

Pruebas de recuperacion de informacion — API de Jira

Campo	Valor
Producto	Plataforma KPIs Jira
Version	1.0
Fecha	15 de mayo de 2026
Relacionado con	UC-07, UC-09, PROJECT_CHARTER.md
Objetivo	Validar que el sistema obtiene, persiste y recupera datos de Jira de forma correcta y resiliente

1. Que significa "recuperacion de informacion" en este proyecto

En este contexto cubre tres niveles:

Nivel	Que se prueba	Ejemplo
A. Recuperacion (fetch)	Llamadas correctas a la API/JQL y datos completos	Search JQL devuelve todos los issues paginados
B. Persistencia	Los datos obtenidos se guardan bien en PostgreSQL	Issue + changelog sin perdida de campos
C. Recuperacion ante fallos	Tras error de red, 429 o caida, el sistema reanuda sin duplicar ni perder	Sync interrumpida en pagina 3 continua en pagina 3

2. Alcance de las pruebas

En alcance

- Autenticacion contra Jira Cloud (API token / Basic).
- Busqueda por JQL (/rest/api/3/search o endpoint vigente).
- Obtencion de issue individual y changelog.
- Paginacion (startAt, maxResults).
- Sync incremental (updated >= ultima_sync).
- Reintentos y manejo de 429 / 5xx.
- Reanudacion de sync incompleta (checkpoint).
- Integridad de conteos (issues recuperados = filas en BD).
- Registro en sync_runs (estado, error, filas).

Fuera de alcance (v1)

- Pruebas de carga masiva (>10k issues) sin acuerdo de volumen.
- Jira Server/Data Center (salvo que se indique).
- Escritura en Jira (create/update issues).

3. Pre-requisitos

#	Requisito
P1	Proyecto sandbox en Jira con >= 20 issues de prueba
P2	API token con permiso Browse projects y lectura de issues
P3	Variables en .env: JIRA_BASE_URL, JIRA_EMAIL, JIRA_API_TOKEN, JIRA_PROJECT_KEY
P4	Catalogo JQL en docs/jql-queries.md (min. 2 consultas)
P5	PostgreSQL accesible (local Docker o staging)
P6	Herramientas: Postman/Insomnia o curl; pytest para automatizados

Datos de prueba recomendados en sandbox

Crear o identificar:

- 5 issues Done en sprint cerrado.
- 5 issues In Progress en sprint abierto.
- 3 issues sin actualizar > 7 dias (aging).
- 1 issue con transiciones múltiples (para changelog).

4. Matriz de pruebas

ID	Tipo	Nombre	Prioridad
TR-01	Funcional	Conexion y autentificacion API	P0
TR-02	Funcional	Busqueda JQL basica	P0
TR-03	Funcional	Paginacion completa	P0
TR-04	Funcional	Issue individual + changelog	P0
TR-05	Funcional	Sync incremental (updated)	P0
TR-06	Integracion	Persistencia en PostgreSQL	P0
TR-07	Recuperacion	Reintento tras timeout/5xx	P0
TR-08	Recuperacion	Manejo rate limit 429	P0
TR-09	Recuperacion	Reanudacion sync interrumpida	P0
TR-10	Integridad	Conteo issues Jira vs BD	P0
TR-11	Funcional	JQL invalido / sin resultados	P1
TR-12	Seguridad	Token invalido o expirado	P1

TR-13	Recuperacion	Modo degradado (ultimo snapshot KPI)	P1
TR-14	Operacion	Registro sync_runs completo	P1
TR-15	Regresion	Idempotencia (re-sync sin duplicados)	P1

5. Casos de prueba detallados

TR-01 — Conexion y autenticacion

Campo	Valor
Objetivo	Verificar credenciales y alcance del token
Endpoint	GET {JIRA_BASE_URL}/rest/api/3/myself

Pasos:

1. Configurar Basic Auth: email:api_token en Base64.
2. Ejecutar GET /rest/api/3/myself.
3. Verificar respuesta 200 y campo accountId.

Resultado esperado:

Status 200; cuerpo con identidad del usuario de integracion.

Fallo:

401 => revisar token/email. | 403 => permisos insuficientes.

Evidencia: captura Postman o log con timestamp (sin mostrar token).

TR-02 — Busqueda JQL basica

Campo	Valor
Objetivo	Recuperar issues con JQL del catalogo
JQL ejemplo	project = {KEY} ORDER BY updated DESC

Pasos:

1. Ejecutar search con maxResults=10.
2. Validar issues[] no vacio (si sandbox tiene datos).
3. Validar campos minimos: key, id, fields.summary, fields.status, fields.updated.

Resultado esperado:

200; array issues coherente con Jira UI para la misma JQL.

TR-03 — Paginacion completa

Campo	Valor
Objetivo	Recuperar todos los issues cuando total > maxResults

Pasos:

1. Primera petición: maxResults=50, startAt=0.
2. Leer total en respuesta.
3. Iterar startAt += 50 hasta startAt >= total.
4. Acumular conteo de issues obtenidos.

Resultado esperado:

Suma de issues de todas las paginas = total de Jira. Sin keys duplicadas entre paginas.

Criterio de fallo:

Detenerse antes de total => bug de paginación en cliente.

TR-04 — Issue individual y changelog

Campo	Valor
Objetivo	Recuperar historial para cycle time

Pasos:

1. GET /rest/api/3/issue/{KEY}?expand=changelog o endpoint de changelog.
2. Verificar transiciones de estado en historial.
3. Tras sync del backend, comprobar tabla issue_changelog (o equivalente) en BD.

Resultado esperado:

Al menos una transición In Progress => Done para issue de prueba preparado.

TR-05 — Sync incremental

Campo	Valor
Objetivo	Solo recuperar issues modificados desde ultima sync

Pasos:

1. Ejecutar sync completa inicial; guardar last_sync_at.
2. Modificar 1 issue en Jira (cambiar summary o estado).
3. Esperar 1 min; ejecutar sync incremental.
4. Verificar JQL incluye filtro temporal, ej.: updated >= '2026-05-15 10:00'.

Resultado esperado:

Solo el issue modificado (y posibles vinculados) se actualiza en BD. sync_runs.rows_updated acorde (no reimportar todo el proyecto).

TR-06 — Persistencia en PostgreSQL

Campo	Valor
Objetivo	Datos recuperados disponibles para KPIs

Pasos:

1. Ejecutar sync via POST /api/v1/sync/trigger (o job).
2. Consultar BD: SELECT COUNT(*) FROM issues WHERE project_key = ?
3. Comparar con conteo TR-03 para mismo alcance JQL.

Resultado esperado:

Conteo BD = conteo API (mismo filtro JQL). Campos obligatorios no NULL: issue_key, status, updated_at.

TR-07 — Reintento tras error 5xx / timeout

Campo	Valor
Objetivo	Recuperacion automatica ante fallo transitorio

Pasos:

1. Simular timeout (mock) o usar proxy que devuelva 503 en 1a llamada.
2. Configurar politica: 3 reintentos, backoff 2s, 4s, 8s.
3. Segunda llamada exitosa.

Resultado esperado:

Sync termina completed (no failed). Logs muestran reintento con numero de intento. No duplicar filas en BD (misma issue_key UPSERT).

TR-08 — Rate limit 429

Campo	Valor
Objetivo	Respetar limites y recuperar sin perder pagina

Pasos:

1. Forzar muchas peticiones seguidas o mock 429 con header Retry-After: 5.
2. Verificar que el cliente espera y reintentra.
3. Completar paginacion.

Resultado esperado:

No failed inmediato; espera >= Retry-After. Checkpoint de startAt conservado.

TR-09 — Reanudacion de sync interrumpida

Campo	Valor
Objetivo	Recuperacion tras caida a mitad de importacion

Pasos:

1. Iniciar sync de proyecto con >= 100 issues.
2. Interrumpir proceso (kill container / Ctrl+C) en pagina 2.
3. Verificar sync_runs.status = interrupted y checkpoint last_start_at=100.
4. Reiniciar sync.

Resultado esperado:

Reanuda en startAt=100 (no desde 0). Estado final completed; total issues correcto.

Si no hay checkpoint (MVP):

Documentar como limitacion; sync incremental por updated evita reimportar todo.

TR-10 — Integridad de conteo

Campo	Valor
-------	-------

Objetivo	Auditoria recuperacion vs fuente
----------	----------------------------------

Pasos:

1. En Jira UI: filtro JQL => conteo issues.
2. En BD: COUNT(*) mismo alcance.
3. Listar diff: keys en Jira no en BD y viceversa.

Resultado esperado:

Diff vacio tras sync exitosa. Informe diff guardado como evidencia de prueba.

TR-11 — JQL invalido / sin resultados

JQL: project = NOEXISTE123

Resultado esperado:

Error controlado o issues=[]; sync completed con 0 filas; mensaje claro en logs/UI.

TR-12 — Token invalido

Resultado esperado:

401; sync failed; no escribir datos parciales corruptos; alerta en panel UC-08.

TR-13 — Modo degradado (KPIs)

Campo	Valor
Precondicion	Ultima sync exitosa hace < 24 h

Pasos:

1. Simular Jira caido (mock 503 en todas las llamadas).
2. Abrir dashboard.

Resultado esperado:

Dashboard muestra ultimos kpi_snapshots + aviso "Datos no actualizados — Jira no disponible".

TR-14 — Registro sync_runs

Campos obligatorios tras cada ejecucion:

Campo	Ejemplo
id	uuid
started_at	timestamp
finished_at	timestamp
status	completed / failed / interrupted
rows_fetched	150
rows_upserted	148
error_message	null o texto

jql_used	project = X AND ...
----------	---------------------

TR-15 — Idempotencia

Pasos:

1. Ejecutar misma sync incremental 2 veces seguidas sin cambios en Jira.

Resultado esperado:

Mismo numero de filas en BD; rows_upserted ~= 0 en segunda ejecucion. Sin duplicados por issue_key UNIQUE.

6. Pruebas manuales rapidas (curl)

Sustituir variables antes de ejecutar.

```
# TR-01 - Autenticacion
curl -s -o /dev/null -w "%{http_code}" \
  -u "EMAIL:API_TOKEN" \
  "https://TU-DOMINIO.atlassian.net/rest/api/3/myself"

# TR-02 - JQL search
curl -s -u "EMAIL:API_TOKEN" \
  -G "https://TU-DOMINIO.atlassian.net/rest/api/3/search/jql" \
  --data-urlencode "jql=project=PROJ ORDER BY updated DESC" \
  --data-urlencode "maxResults=10" \
  | jq '.issues | length, .total // .issues.total'
```

7. Pruebas automatizadas (estructura pytest)

Cuando exista backend/, usar esta estructura:

```
backend/tests/
  conftest.py           # fixtures: jira_client, db_session
  test_jira_auth.py      # TR-01, TR-12
  test_jira_search.py    # TR-02, TR-03, TR-11
  test_jira_changelog.py # TR-04
  test_sync_incremental.py # TR-05, TR-15
  test_sync_recovery.py  # TR-07, TR-08, TR-09
  test_sync_persistence.py # TR-06, TR-10, TR-14
```

8. Criterios de aceptacion global (go-live)

#	Criterio
G1	TR-01 a TR-06 pasan en staging
G2	TR-07 a TR-10 pasan (reintentos y reanudacion documentados)
G3	TR-14 siempre poblado en cada sync
G4	Evidencia TR-10 (informe diff vacio) archivada

G5	Runbook describe que hacer si TR-08 o TR-12 fallan en prod
----	--

9. Plantilla de registro de ejecucion

Campo	Valor
Fecha	
Entorno	local / staging
Ejecutor	
Version commit	
TR ejecutados	TR-01 ...
Resultado	PASS / FAIL
Incidencias	
Evidencia	enlace captura / log

10. Trazabilidad

Prueba	Caso de uso	Objetivo charter
TR-01 a TR-04	UC-09	O1 — Backend API Jira
TR-05 a TR-06, TR-10	UC-09, UC-10	O2 — KPIs
TR-07 a TR-09, TR-13	UC-07, UC-08	O5 — Operacion
TR-14	UC-08	Transparencia sync

11. Historial

Version	Fecha	Cambios
1.0	2026-05-15	Plan inicial TR-01 a TR-15